accidentally or intentionally put your code in an infinite loop—the simulator freezes, looping forever, and there is no way you can stop it except by killing it. The interface does not work in such a case. Running the interface and the execution module in separate threads would help, I guess.

- A minor glitch detected while boundary testing is that if you attempt to enter a negative program-load address, or an address greater than *ffffh* or 65535 in decimal, the simulator does not complain, but automatically detects an invalid range and loads the assembled code at the default loading address, *4200h*. But consider this scenario: you load code at *ffff0h*, and the length of your code is more than 15 (which will try to go beyond the *ffffh* memory limit). With this, if you assemble/run the code, the simulator immediately crashes. This should wrap around memory and continue to be loaded and executed from *0000h* after reaching the limit *ffffh*, like in the real machine, or simply notify the user about this and stop.
- Although this is a minor bug, I think it is worth pointing it out: if you enter an invalid decimal value in the decimal text box, the hexadecimal conversion shows an invalid value in the hexadecimal box. I think giving some warning of an invalid input would help users.

Version 1.3.7 vs 1.3.5

Version 1.3.7 came out recently, almost one year after the last one. On the official website, the 1.3.7 Windows binary (with and without GTK runtime) and the 1.3.7 source code is available. The .deb is obsolete. There are not many externally visible differences in these two versions, but some nasty bugs were killed.

Version 1.3.6 was released in March 2010, but did not fix the real bugs in 1.3.5—like in 1.3.5, clicking '*Help->About*' would crash the program immediately. If you changed source code in an open file and clicked *new*, loaded another file, or simply closed it, there was no prompt to save the document. These were fixed in 1.3.7.

The really nasty DAA bug was killed in 1.3.7, for which reason you should avoid using 1.3.5. In 1.3.5, if you ran DAA after adding 99h and 01h, the accumulator ought to have contained 00h, and the carry flag should have been set, but though the accumulator was 00h, the carry flag would not get set. Try adding 59h and 45h and then apply DAA. The result would be in an invalid BCD form. Thankfully, this nasty bug was killed in 1.3.7. I would like to make you aware that there are a lot of simulators that have implemented DAA incorrectly, so please confirm DAA operation with the above or a similar operation as operating in boundary conditions, or with the real machine output.

Some other bugs have been fixed as well, so I recommend you download the latest version 1.3.7 if it is available from your distro repository, or if not, install from source.

Have a look at the version release notes: *https://launchpad.net/gnusim8085/+announcement/7810*.

Still to implement

- The RST instructions (a 1-byte call instruction, jumping to a pre-determined jump location).
- RIM and SIM instructions—you would not be able to test any interrupt or serial I/O-related code. You would notice the '*Int-reg*' register in the simulator interface, which apparently represents the interrupt register that holds the masks, the pending interrupts, etc. This is non-functional.
- There are problems when printing on Windows, and the Linux environment implementation is very primitive.
- If you look at the op-codes, you will notice that there are 10 unused ones that have no mnemonics associated. These are undocumented instructions. They are not documented because they are not supported by 8086 and higher processors. These instructions and the related undocumented flags are not supported by the simulator. This will not bother many people, since, being 'undocumented', they are not generally used. Implementation of these is not that urgent, but the availability of the undocumented instructions would definitely make this simulator special and draw attention as these instructions can make some composite operation with a single instruction.

What you need to remember...

This is essentially a very good simulator to work with for educational purposes. If you have to practice code for your college exam, or to test the code in the practical copies, then I would recommend this simulator. Some of my friends and I have used it during our undergraduate course, and had no problems. It has a very simple and clutter-free interface, an assembler, and the best part is that being under GPL 3, you can see the code and even edit it if you like—or request any feature update from the developers.

Programming in the simulator is easy, but it should be kept in mind that if you are doing a course, you need to hand-assemble and enter the code when presented with the actual 8085 in your college syllabus. It is good to make a habit of hand-assembling and directly entering code on the actual machine, to keep in touch. END

References

- Official Site: *http://gnusim8085.org/index.php*
- 8085 Wiki: *http://en.wikipedia.org/wiki/Intel_8085*
- GNUSim8085 Wiki: *http://en.wikipedia.org/wiki/GNUSim8085*
- GNUSim8085 Announcements: *https://launchpad.net/gnusim8085/+announcements*
- 'Microprocessor Architecture, Programming, and Applications with the 8085' by Ramesh Gaonkar

By: Arjun Pakrashi

The author is a Computer Science enthusiast and passionate Fedora user. He blogs at: http://phoxis.org and can be reached at: phoxis@gmail.com